Pochak OTT Platform - API Specification

> *Version: 1.0.0 Last Updated: 2026-03-26 Base URL: `http://localhost:8080/api/v1` (Gateway)*

Table of Contents

1. Overview
2. Authentication
3. Common Response Format
4. Identity Service
5. Content Service
6. Commerce Service
7. Operation Service
8. Admin Service
9. Gateway
10. Policy Compliance Matrix

---

1. Overview

Pochak is a microservice-based OTT platform for sports content. The system is composed of 6 backend services, all exposed through a single API Gateway.

Service	Port	Path Prefix	Description
Gateway	8080	`/api/v1/*`, `/admin/*`	JWT validation, routing, rate limiting, CORS
Identity	8081	`/api/v1/auth`, `/api/v1/users`, `/api/v1/guardians`	Authentication, user management, guardian
Content	8082	`/api/v1/contents`, `/api/v1/sports`, `/api/v1/teams`, etc.	Media assets, social, search, streaming
Commerce	8083	`/api/v1/products`, `/api/v1/purchases`, `/api/v1/wallet`, etc.	Products, purchases, entitlements, wallet
Operation	8084	`/api/v1/venues`, `/api/v1/cameras`, `/api/v1/reservations`, etc.	Venues, cameras, reservations, ingest
Admin	8085	`/admin/api/v1/*`	BO admin RBAC, site management, CS, analytics

All client requests enter through the Gateway at port 8080. The Gateway strips the `/api/v1` prefix before forwarding to downstream services (except Admin Service, which receives paths as-is).

---

2. Authentication

2.1 JWT Flow

All protected endpoints require a JWT access token in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

The Gateway validates the JWT, strips any incoming `X-User-Id` and `X-User-Role` headers (SEC-001), and injects verified values extracted from the JWT claims into downstream requests.

Token Claims:

Claim	Description
`sub`	User ID (Long)
`role`	User role (USER, ADMIN)
`exp`	Expiration timestamp

2.2 OAuth2 PKCE Flow (Mobile)

Mobile apps use OAuth2 Authorization Code flow with PKCE (SEC-003):

1. **Initiate**: `GET /api/v1/auth/oauth2/authorize/{provider}?code_challenge=...&code_challenge_method=S256&platform=mobile`
2. **Callback**: Provider redirects to `GET /api/v1/auth/oauth2/callback/{provider}?code=...&state=...`
3. **Exchange**: `POST /api/v1/auth/oauth2/token` with `{ code, codeVerifier }`

Auth codes are single-use and expire after 30 seconds (SEC-006).

2.3 Token Refresh

```
POST /api/v1/auth/refresh
Body: { "refreshToken": "..." }
Response: { "accessToken": "...", "refreshToken": "...", "expiresIn": 3600 }
```

2.4 Token Blacklist

On logout, the user's token is added to a Redis blacklist (`token-blacklist:{userId}`). The Gateway checks this list on every request (SEC-002).

2.5 Auth Level Definitions

Level	Description
PUBLIC	No JWT required. If JWT is provided, user context is extracted but failure is ignored.
USER	Valid JWT required. `X-User-Id` header injected.
ADMIN	Valid JWT with `role=ADMIN` required. Enforced at Gateway (SEC-004).

---

3. Common Response Format

3.1 Success Response (`ApiResponse<T>`)

```
{
  "success": true,
  "data": { ... },
  "meta": {                    // present only for paginated responses
    "page": 0,
    "size": 20,
    "totalCount": 150,
    "totalPages": 8
  }
}
```

3.2 Error Response

```
{
  "success": false,
  "error": {
    "code": "ENTITY_NOT_FOUND",
    "message": "User not found with id: 123"
  }
}
```

3.3 Common Error Codes

Code	HTTP Status	Description
`INVALID_INPUT`	400	Request validation failed
`UNAUTHORIZED`	401	Missing or invalid JWT
`FORBIDDEN`	403	Insufficient permissions
`ENTITY_NOT_FOUND`	404	Resource not found
`DUPLICATE_ENTITY`	409	Unique constraint violated
`TOO_MANY_REQUESTS`	429	Rate limit exceeded
`INTERNAL_SERVER_ERROR`	500	Unexpected server error

3.4 Pagination Parameters

Most list endpoints accept Spring Pageable parameters:

Parameter	Type	Default	Description
`page`	int	0	Zero-based page index
`size`	int	20	Page size
`sort`	string	varies	Sort field and direction (e.g., `createdAt,desc`)

---

4. Identity Service

Base path (via Gateway): `/api/v1` Internal port: 8081

4.1 Auth - Login & Session Management

POST `/auth/login`

> *일반 로그인 / Standard login*

Auth	PUBLIC
Request Body	`{ "loginId": string, "password": string }`
Response	`TokenResponse { accessToken, refreshToken, expiresIn, tokenType }`
Status	200 OK, 401 Unauthorized

POST `/auth/refresh`

> *토큰 갱신 / Token refresh*

Auth	PUBLIC
Request Body	`{ "refreshToken": string }`
Response	`TokenResponse`
Status	200 OK, 400 Bad Request

POST `/auth/social`

> *소셜 로그인 / Social login (direct token exchange)*

Auth	PUBLIC
Request Body	`SocialLoginRequest { provider, accessToken }`
Response	`TokenResponse`
Status	200 OK

POST `/auth/logout`

> *로그아웃 / Logout (blacklists token in Redis)*

Auth	USER
Headers	`X-User-Id` (injected by Gateway)
Response	`null`
Status	200 OK

DELETE `/auth/withdraw`

> *회원 탈퇴 / Account withdrawal (publishes UserWithdrawnEvent)*

Auth	USER
Headers	`X-User-Id`
Response	`null`
Status	200 OK

4.2 Auth - Signup (4 Routes)

POST `/auth/signup`

> *Route A: 국내 성인 회원가입 / Domestic adult (14+) signup*

Auth	PUBLIC
Request Body	`DomesticSignupRequest { loginId, password, name, phone, email?, birthday, gender?, verifiedToken, consents[] }`
Response	`TokenResponse`
Status	200 OK, 400 Validation error

POST `/auth/signup/minor`

> *Route B: 국내 미성년(14세 미만) 회원가입 / Domestic minor signup*

Auth	PUBLIC
Request Body	`MinorSignupRequest { loginId, password, name, phone, birthday, guardianVerifiedToken, verifiedToken, consents[] }`
Response	`TokenResponse`
Status	200 OK

POST `/auth/signup/social`

> *Route C: SNS 회원가입 / Social signup*

Auth	PUBLIC
Request Body	`SocialSignupRequest { provider, accessToken, nickname, phone?, verifiedToken?, consents[] }`
Response	`TokenResponse`
Status	200 OK

POST `/auth/signup/foreign`

> *Route D: 해외 사용자 회원가입 / Foreign user signup*

Auth	PUBLIC
Request Body	`ForeignSignupRequest { loginId, password, name, email, nationality?, consents[] }`
Response	`TokenResponse`
Status	200 OK

4.3 Auth - Phone Verification

POST `/auth/phone/send-code`

> *전화번호 인증 코드 발송 / Send phone verification code*

Auth	PUBLIC
Request Body	`{ "phone": string, "purpose": string }`
Response	`{ "sent": true, "expiresIn": 180 }`
Status	200 OK

POST `/auth/phone/verify-code`

> *인증 코드 확인 / Verify phone code*

Auth	PUBLIC
Request Body	`{ "phone": string, "code": string, "purpose": string }`

Response	`{ "verified": true, "verifiedToken": string }`
Status	200 OK

GET `/auth/phone/check`

> *전화번호 등록 여부 확인 / Check if phone is registered*

Auth	PUBLIC
Query	`phone` (string)
Response	`{ "registered": boolean, "maskedLoginId"?: string }`
Status	200 OK

4.4 Auth - Guardian Verification

POST `/auth/guardian/verify`

> *보호자 인증 / Verify guardian (for minor signup)*

Auth	PUBLIC
Query	`guardianVerifiedToken` (string)
Response	`{ "verified": true, "guardianConsentToken": string }`
Status	200 OK

4.5 Auth - Duplicate Check

GET `/auth/check-duplicate`

> *중복 확인 / Check duplicate loginId, email, or phone*

Auth	PUBLIC
Query	`loginId?`, `email?`, `phone?` (at least one required)
Response	`CheckDuplicateResponse { field, value, available }`
Status	200 OK

4.6 OAuth2 Flow

GET `/auth/oauth2/authorize/{provider}`

> *OAuth2 인증 시작 / Initiate OAuth2 with optional PKCE*

Auth	PUBLIC
Path	`provider`: kakao, google, naver
Query	`code_challenge?` (required for mobile), `code_challenge_method?` (S256), `platform?` (mobile/web)
Response	302 Redirect to OAuth provider
Status	302, 400 Bad Request

GET `/auth/oauth2/callback/{provider}`

> *OAuth2 콜백 / OAuth2 callback (redirects to app/web with auth code or signup token)*

Auth	PUBLIC
Path	`provider`: kakao, google, naver
Query	`code`, `state?`
Response	302 Redirect to `pochak://auth?code=...` (mobile) or web URL
Status	302

POST `/auth/oauth2/token`

> *인증 코드 교환 / Exchange one-time auth code for tokens (SEC-006)*

Auth	PUBLIC
Request Body	`{ "code": string, "codeVerifier"?: string }`
Response	`TokenResponse`
Status	200 OK, 400 Invalid/expired code, 401 PKCE failure

POST `/auth/oauth2/complete-signup`

> *OAuth 회원가입 완료 / Complete OAuth signup*

Auth	PUBLIC
Request Body	`{ "signupToken": string, "nickname": string }`
Response	`TokenResponse`
Status	200 OK

POST `/auth/oauth2/link`

> *OAuth 계정 연동 / Link OAuth provider to existing account*

Auth	PUBLIC
Request Body	`{ "signupToken": string }`
Response	`TokenResponse`
Status	200 OK

4.7 User Profile & Preferences

GET `/users/me`

> *내 프로필 조회 / Get my profile*

Auth	USER

Response	`UserProfileResponse { id, username, email, phone, name, birthday, gender, nationality, profileImage, status, isMarketing, isAge14Above }`
Status	200 OK

PUT `/users/me`

> *내 프로필 수정 / Update my profile*

Auth	USER
Request Body	`UpdateProfileRequest { name?, email?, profileImage?, gender? }`
Response	`UserProfileResponse`
Status	200 OK

GET `/users/me/preferences`

> *내 선호 정보 조회 / Get my preferences*

Auth	USER
Response	`UserPreferencesResponse { preferredSports[], preferredAreas[], usagePurpose }`
Status	200 OK

PUT `/users/me/preferences`

> *내 선호 정보 수정 / Update my preferences*

Auth	USER
Request Body	`UpdatePreferencesRequest { preferredSports[], preferredAreas[], usagePurpose? }`
Response	`UserPreferencesResponse`
Status	200 OK

GET `/users/me/status`

> *내 계정 상태 조회 / Get my account status*

Auth	USER
Response	`UserStatusResponse { status, statusHistory[] }`
Status	200 OK

4.8 Guardian Management

POST `/guardians/request`

> *보호자 연결 요청 / Request guardian connection*

Auth	USER
Request Body	`{ "minorId": Long, "consentMethod": string }`

Response	`GuardianResponseDto`
Status	200 OK

POST `/guardians/{id}/verify`

> *보호자 인증 완료 / Verify guardian connection*

Auth	USER
Path	`id`: guardian relationship ID
Query	`verificationToken?`
Response	`GuardianResponseDto`
Status	200 OK

GET `/guardians/minors`

> *보호자의 미성년자 목록 / List minors for guardian*

Auth	USER
Response	`List<GuardianResponseDto>`
Status	200 OK

GET `/guardians/my-guardian`

> *미성년자의 보호자 조회 / Get my guardian info*

Auth	USER
Response	`GuardianResponseDto`
Status	200 OK

PUT `/guardians/{id}/limit`

> *월간 결제 한도 설정 / Update monthly payment limit*

Auth	USER
Request Body	`{ "monthlyPaymentLimit": Integer }`
Response	`GuardianResponseDto`
Status	200 OK

DELETE `/guardians/{id}`

> *보호자 관계 해제 / Revoke guardian relationship*

Auth	USER
Response	`null`

Status	200 OK

GET `/guardians/check-limit`

> *결제 한도 확인 (내부) / Check payment limit (internal, called by Commerce)*

Auth	INTERNAL
Query	`minorId` (Long), `amount` (Integer)
Response	`PaymentLimitCheckDto { allowed, remainingLimit, monthlyLimit }`
Status	200 OK

4.9 Admin Member Management

GET `/admin/members`

> *회원 목록 조회 / List members (admin)*

Auth	ADMIN
Query	`page`, `size`, `status?`, `role?`, `search?`, `searchType?`
Response	`AdminMemberListResponse { members[], totalCount }`
Status	200 OK

GET `/admin/members/{id}`

> *회원 상세 조회 / Get member detail (admin)*

Auth	ADMIN
Response	`AdminMemberResponse`
Status	200 OK, 404

PUT `/admin/members/{id}/status`

> *회원 상태 변경 / Update member status (admin)*

Auth	ADMIN
Request Body	`UpdateMemberStatusRequest { status, reason? }`
Response	`AdminMemberResponse`
Status	200 OK

PUT `/admin/members/{id}/role`

> *회원 역할 변경 / Update member role (admin)*

Auth	ADMIN
Request Body	`UpdateMemberRoleRequest { role }`

Response	`AdminMemberResponse`
Status	200 OK

---

5. Content Service

Base path (via Gateway): `/api/v1` Internal port: 8082

5.1 Home

GET `/home`

> *홈 피드 / Home feed (banners, sections, recommended content)*

Auth	PUBLIC
Response	`HomeResponse { banners[], sections[], liveNow[], popular[], recommended[] }`
Status	200 OK

5.2 Sports

GET `/sports`

> *종목 목록 / List sports*

Auth	PUBLIC
Query	`isActive?` (boolean), pagination
Response	`List<SportListResponse>` with `PageMeta`
Status	200 OK

GET `/sports/{id}`

> *종목 상세 / Sport detail*

Auth	PUBLIC
Response	`SportDetailResponse`
Status	200 OK, 404

POST `/sports`

> *종목 생성 / Create sport*

Auth	ADMIN
Request Body	`CreateSportRequest { name, code, imageUrl?, displayOrder }`
Response	`SportDetailResponse`
Status	201 Created

PUT `/sports/{id}`

> *종목 수정 / Update sport*

Auth	ADMIN
Request Body	`UpdateSportRequest { name?, code?, imageUrl?, displayOrder?, isActive? }`
Response	`SportDetailResponse`
Status	200 OK

PUT `/sports/order`

> *종목 표시 순서 변경 / Update display order*

Auth	ADMIN
Request Body	`UpdateDisplayOrderRequest { orders: [{id, displayOrder}] }`
Response	`null`
Status	200 OK

DELETE `/sports/{id}`

> *종목 삭제 / Delete sport*

Auth	ADMIN
Status	204 No Content

GET `/sports/{sportId}/tags`

> *종목 태그 목록 / List sport tags*

Auth	PUBLIC
Response	`List<SportTagResponse>`

POST `/sports/{sportId}/tags`

> *종목 태그 생성 / Create sport tag*

Auth	ADMIN
Request Body	`CreateSportTagRequest { name, englishName?, imageUrl?, displayOrder }`
Response	`SportTagResponse`
Status	201 Created

PUT `/sports/{sportId}/tags/{tagId}`

> *종목 태그 수정 / Update sport tag*

Auth	ADMIN
Response	`SportTagResponse`

DELETE `/sports/{sportId}/tags/{tagId}`

> *종목 태그 삭제 / Delete sport tag*

Auth	ADMIN
Status	204 No Content

5.3 Teams

GET `/teams`

> *팀 목록 / List teams by sport*

Auth	PUBLIC
Query	`sportId` (required)
Response	`List<Team>`

GET `/teams/{id}`

> *팀 상세 / Team detail*

Auth	PUBLIC
Response	`Team`

5.4 Competitions

GET `/competitions`

> *대회 목록 / List competitions*

Auth	PUBLIC
Query	`sportId?`, `status?` (UPCOMING/ONGOING/FINISHED), `isDisplayed?`, `keyword?`, pagination
Response	`List<CompetitionListResponse>` with `PageMeta`

GET `/competitions/{id}`

> *대회 상세 / Competition detail*

Auth	PUBLIC
Response	`CompetitionDetailResponse`

POST `/competitions`

> *대회 생성 / Create competition*

Auth	ADMIN
Request Body	`CreateCompetitionRequest`
Response	`CompetitionDetailResponse`

Status	201 Created

PUT `/competitions/{id}`

> *대회 수정 / Update competition*

Auth	ADMIN
Response	`CompetitionDetailResponse`

DELETE `/competitions/{id}`

> *대회 삭제 / Delete competition*

Auth	ADMIN
Status	204 No Content

POST `/competitions/access`

> *비공개 대회 초대코드 입력 / Access private competition via invite code*

Auth	USER
Query	`inviteCode`, `userId`
Response	`CompetitionDetailResponse`
Status	200 OK, 429 Rate limited

GET `/competitions/visited`

> *방문한 대회 목록 / Get visited competitions*

Auth	USER
Query	`userId`
Response	`List<CompetitionListResponse>`

5.5 Matches

GET `/matches`

> *경기 목록 / List matches*

Auth	PUBLIC
Query	`competitionId?`, `sportId?`, `venueId?`, `status?`, `isDisplayed?`, `dateFrom?`, `dateTo?`, pagination
Response	`List<MatchListResponse>` with `PageMeta`

GET `/matches/{id}`

> *경기 상세 / Match detail*

Auth	PUBLIC
Response	`MatchDetailResponse`

POST `/matches`

> *경기 생성 / Create match*

Auth	ADMIN
Request Body	`CreateMatchRequest`
Response	`MatchDetailResponse`
Status	201 Created

PUT `/matches/{id}`

> *경기 수정 / Update match*

Auth	ADMIN
Response	`MatchDetailResponse`

PUT `/matches/{id}/status`

> *경기 상태 변경 / Change match status (SCHEDULED -> LIVE -> FINISHED)*

Auth	ADMIN
Request Body	`ChangeMatchStatusRequest { status }`
Response	`MatchDetailResponse`

DELETE `/matches/{id}`

> *경기 삭제 / Delete match*

Auth	ADMIN
Status	204 No Content

5.6 Organizations

GET `/organizations`

> *단체 목록 / List organizations*

Auth	PUBLIC
Query	`orgType?` (PRIVATE/PUBLIC), `parentId?`, `sportId?`, `keyword?`, pagination
Response	`List<OrganizationListResponse>` with `PageMeta`

GET `/organizations/{id}`

> *단체 상세 / Organization detail*

Auth	PUBLIC
Response	`OrganizationDetailResponse`

POST `/organizations`

> *단체 생성 / Create organization*

Auth	ADMIN
Request Body	`CreateOrganizationRequest`
Response	`OrganizationDetailResponse`
Status	201 Created

PUT `/organizations/{id}`

> *단체 수정 / Update organization*

Auth	ADMIN
Response	`OrganizationDetailResponse`

DELETE `/organizations/{id}`

> *단체 삭제 / Delete organization*

Auth	ADMIN
Status	204 No Content

GET `/organizations/{id}/children`

> *하위 단체 목록 / List child organizations*

Auth	PUBLIC
Response	`List<OrganizationListResponse>`

5.7 Clubs

GET `/clubs/nearby`

> *주변 클럽 / Nearby clubs*

Auth	PUBLIC
Query	`sportId?`, `siGunGuCode?`, `lat?`, `lng?`, pagination
Response	`List<ClubListResponse>` with `PageMeta`

GET `/clubs/popular`

> *인기 클럽 / Popular clubs*

Auth	PUBLIC
Query	pagination
Response	`List<ClubListResponse>` with `PageMeta`

GET `/clubs/recent`

> *최신 클럽 / Recently created clubs*

Auth	PUBLIC
Response	`List<ClubListResponse>` with `PageMeta`

GET `/clubs/{teamId}`

> *클럽 상세 / Club detail*

Auth	PUBLIC
Response	`ClubDetailResponse`

POST `/clubs/{teamId}/join`

> *클럽 가입 / Join club*

Auth	USER
Request Body	`JoinClubRequest { userId, nickname? }`
Response	`MembershipResponse`
Status	201 Created

GET `/clubs/{teamId}/members`

> *클럽 멤버 / Club members list*

Auth	PUBLIC
Response	`List<ClubMemberResponse>`

5.8 Content Assets - Live

GET `/contents/live`

> *라이브 목록 / List live assets*

Auth	PUBLIC
Query	`ownerType?`, `venueId?`, `dateFrom?`, `dateTo?`, `isDisplayed?`, `visibility?`, pagination
Response	`List<LiveAssetListResponse>` with `PageMeta`

GET `/contents/live/{id}`

> *라이브 상세 / Live asset detail*

Auth	PUBLIC
Response	`LiveAssetDetailResponse`

POST `/contents/live`

> *라이브 생성 / Create live asset*

Auth	ADMIN
Request Body	`CreateLiveAssetRequest`
Response	`LiveAssetDetailResponse`
Status	201 Created

PUT `/contents/live/{id}`

> *라이브 수정 / Update live asset*

Auth	ADMIN
Response	`LiveAssetDetailResponse`

PUT `/contents/live/bulk-visibility`

> *라이브 일괄 공개 설정 / Bulk update live visibility*

Auth	ADMIN
Request Body	`BulkVisibilityRequest { ids[], visibility }`
Response	`null`

DELETE `/contents/live/{id}`

> *라이브 삭제 / Delete live asset*

Auth	ADMIN
Status	204 No Content

5.9 Content Assets - VOD

GET `/contents/vod`

> *VOD 목록 / List VOD assets*

Auth	PUBLIC
Query	`ownerType?`, `venueId?`, `dateFrom?`, `dateTo?`, `isDisplayed?`, `visibility?`, pagination
Response	`List<VodAssetListResponse>` with `PageMeta`

GET `/contents/vod/{id}`

> *VOD 상세 / VOD asset detail*

Auth	PUBLIC
Response	`VodAssetDetailResponse`

POST `/contents/vod`

> *VOD 생성 / Create VOD asset*

Auth	ADMIN
Request Body	`CreateVodAssetRequest`
Response	`VodAssetDetailResponse`
Status	201 Created

PUT `/contents/vod/{id}`

> *VOD 수정 / Update VOD asset*

Auth	ADMIN
Response	`VodAssetDetailResponse`

PUT `/contents/vod/bulk-visibility`

> *VOD 일괄 공개 설정 / Bulk update VOD visibility*

Auth	ADMIN
Request Body	`BulkVisibilityRequest { ids[], visibility }`

DELETE `/contents/vod/{id}`

> *VOD 삭제 / Delete VOD asset*

Auth	ADMIN
Status	204 No Content

5.10 Content Assets - Clips

GET `/contents/clips`

> *클립 목록 / List clip assets*

Auth	PUBLIC
Query	`sourceType?`, `visibility?`, `matchId?`, `creatorUserId?`, `isDisplayed?`, pagination
Response	`List<ClipAssetListResponse>` with `PageMeta`

GET `/contents/clips/{id}`

> *클립 상세 / Clip asset detail*

Auth	PUBLIC
Response	`ClipAssetDetailResponse`

POST `/contents/clips`

> *클립 생성 / Create clip asset*

Auth	USER
Request Body	`CreateClipAssetRequest`
Response	`ClipAssetDetailResponse`
Status	201 Created

POST `/contents/clips/create-from-range`

> *구간으로 클립 생성 / Create clip from time range*

Auth	USER
Request Body	`CreateClipFromRangeRequest { sourceAssetId, sourceType, startTimeSec, endTimeSec, title? }`
Response	`ClipAssetDetailResponse`
Status	201 Created

PUT `/contents/clips/{id}`

> *클립 수정 / Update clip asset*

Auth	USER
Response	`ClipAssetDetailResponse`

PUT `/contents/clips/bulk-visibility`

> *클립 일괄 공개 설정 / Bulk update clip visibility*

Auth	ADMIN
Request Body	`BulkVisibilityRequest`

DELETE `/contents/clips/{id}`

> *클립 삭제 / Delete clip asset*

Auth	USER (owner) / ADMIN
Status	204 No Content

5.11 Content Tags

GET `/contents/tags`

> *콘텐츠 태그 조회 / List tags for an asset*

Auth	PUBLIC
Query	`assetType` (live/vod/clip), `assetId` (Long)
Response	`List<AssetTagResponse>`

POST `/contents/tags`

> *콘텐츠 태그 추가 / Create tag*

Auth	ADMIN
Request Body	`CreateAssetTagRequest { assetType, assetId, tagName }`
Response	`AssetTagResponse`
Status	201 Created

DELETE `/contents/tags/{id}`

> *콘텐츠 태그 삭제 / Delete tag*

Auth	ADMIN
Status	204 No Content

5.12 Content Access Control (ABAC)

GET `/contents/{type}/{id}/access`

> *콘텐츠 접근 권한 확인 / Check content access (ABAC evaluation)*

Auth	PUBLIC (userId optional)
Path	`type`: live/vod/clip; `id`: content ID
Query	`userId?`
Response	`AccessCheckResponse { hasAccess, reason, visibility, requiredMembership? }`

POST `/contents/{type}/{id}/acl`

> *ACL 설정 / Set ACL policy for content*

Auth	ADMIN
Request Body	`SetAclRequest { visibility, allowedOrgIds?, deniedOrgIds?, requireSubscription? }`
Response	`VideoAclResponse`
Status	201 Created

GET `/contents/{type}/{id}/acl`

> *ACL 조회 / Get current ACL for content*

Auth	ADMIN

Response	`VideoAclResponse`

5.13 Player

GET `/contents/live/{id}/player`

> *라이브 플레이어 정보 / Live player detail*

Auth	PUBLIC
Response	`PlayerDetailResponse { contentInfo, streamUrl, cameras[], qualityLevels[], matchInfo?, highlights[] }`

GET `/contents/vod/{id}/player`

> *VOD 플레이어 정보 / VOD player detail*

Auth	PUBLIC
Response	`PlayerDetailResponse`

GET `/contents/clips/{id}/player`

> *클립 플레이어 정보 / Clip player detail*

Auth	PUBLIC
Response	`PlayerDetailResponse`

5.14 Streaming

GET `/contents/{contentType}/{contentId}/stream`

> *콘텐츠 스트리밍 URL 조회 / Get playback URL (primary endpoint, ABAC enforced)*

Auth	PUBLIC (user context used for ABAC)
Path	`contentType`: live/vod/clip
Headers	`X-User-Id` (optional)
Response	`PlaybackResponse { streamUrl, protocol, qualityLevels[] }`
Status	200 OK, 403 Access denied

GET `/streaming/{contentType}/{contentId}`

> *스트리밍 정보 / Stream info (legacy low-level)*

Auth	PUBLIC
Response	`StreamInfo { url, protocol, isLive }`

GET `/streaming/cameras/{matchId}`

> *사용 가능 카메라 목록 / Available cameras for match*

Auth	PUBLIC
Response	`List<CameraView>`

GET `/streaming/quality`

> *화질 목록 / Quality levels for stream URL*

Auth	PUBLIC
Query	`url` (string)
Response	`List<QualityLevel>`

5.15 Highlights

GET `/contents/{type}/{id}/highlights`

> *하이라이트 목록 / List highlights*

Auth	PUBLIC
Response	`List<HighlightResponse>`

POST `/contents/{type}/{id}/highlights/detect`

> *하이라이트 자동 탐지 / Auto-detect highlights (stub)*

Auth	ADMIN
Response	`{ "highlights": [...], "generatedClipIds": [...] }`

POST `/contents/{type}/{id}/highlights`

> *수동 하이라이트 생성 / Create manual highlight*

Auth	ADMIN
Request Body	`CreateHighlightRequest { startTimeSec, endTimeSec, label?, description? }`
Response	`HighlightResponse`
Status	201 Created

5.16 Timeline Events

GET `/contents/{type}/{id}/timeline-events`

> *타임라인 이벤트 목록 / List timeline events*

Auth	PUBLIC
Response	`List<TimelineEventResponse>`

POST `/contents/{type}/{id}/timeline-events`

> *타임라인 이벤트 생성 / Create timeline event*

Auth	ADMIN
Request Body	`CreateTimelineEventRequest { timeSec, eventType, label, description? }`
Response	`TimelineEventResponse`
Status	201 Created

5.17 Comments

GET `/contents/{type}/{id}/comments`

> *댓글 목록 / List comments*

Auth	PUBLIC
Query	pagination
Response	`List<CommentResponse>` with `PageMeta`

POST `/contents/{type}/{id}/comments`

> *댓글 작성 / Add comment*

Auth	USER
Request Body	`CreateCommentRequest { userId, content, parentId? }`
Response	`CommentResponse`
Status	201 Created

DELETE `/comments/{id}`

> *댓글 삭제 / Delete comment*

Auth	USER (owner)
Query	`userId`
Status	204 No Content

GET `/comments/{id}/replies`

> *대댓글 조회 / Get replies to comment*

Auth	PUBLIC
Response	`List<CommentResponse>`

5.18 Likes

POST `/contents/{type}/{id}/like`

> *좋아요 토글 / Toggle like*

Auth	USER

Response	`null`

DELETE `/contents/{type}/{id}/like`

> *좋아요 취소 / Remove like*

Auth	USER
Response	`null`

GET `/contents/{type}/{id}/like-count`

> *좋아요 수 조회 / Get like count*

Auth	USER
Response	`LikeCountResponse { count, isLikedByMe }`

5.19 Follows

POST `/follows`

> *팔로우 / Follow target*

Auth	USER
Request Body	`FollowRequest { followerUserId, targetType, targetId }`
Response	`FollowResponse`
Status	201 Created

DELETE `/follows`

> *언팔로우 / Unfollow*

Auth	USER
Query	`followerUserId`, `targetType`, `targetId`
Status	204 No Content

GET `/follows/following`

> *내가 팔로우하는 대상 / List following*

Auth	USER
Query	`userId`
Response	`List<FollowResponse>`

GET `/follows/count`

> *팔로워 수 / Follower count for target*

Auth	PUBLIC
Query	`targetType`, `targetId`
Response	`FollowCountResponse { followerCount }`

5.20 Memberships

POST `/memberships`

> *멤버십 생성 / Create membership*

Auth	ADMIN
Request Body	`CreateMembershipRequest`
Response	`MembershipResponse`
Status	201 Created

POST `/memberships/join`

> *단체 가입 신청 / Join organization (OPEN: auto-approved, CLOSED: pending)*

Auth	USER
Request Body	`JoinOrganizationRequest { userId, organizationId, nickname? }`
Response	`MembershipResponse`
Status	201 Created

GET `/memberships`

> *멤버십 조회 / List memberships*

Auth	USER
Query	`userId?`, `targetType?`, `targetId?`
Response	`List<MembershipResponse>`

GET `/memberships/pending`

> *대기 중인 가입 신청 / List pending memberships (manager)*

Auth	USER (manager)
Query	`orgId`
Response	`List<MembershipResponse>`

PUT `/memberships/{id}`

> *멤버십 역할 변경 / Update membership role*

Auth	USER (manager)

Request Body	`UpdateMembershipRoleRequest { role }`
Response	`MembershipResponse`

PUT `/memberships/{id}/approve`

> *가입 승인 / Approve membership*

Auth	USER (manager)
Request Body	`ApproveMembershipRequest { managerId }`
Response	`MembershipResponse`

PUT `/memberships/{id}/reject`

> *가입 거절 / Reject membership*

Auth	USER (manager)
Request Body	`RejectMembershipRequest { managerId, reason }`
Response	`MembershipResponse`

DELETE `/memberships/{id}`

> *멤버십 삭제 / Delete membership*

Auth	USER (owner/manager)
Status	204 No Content

5.21 Communities

GET `/communities/posts`

> *커뮤니티 게시글 목록 / List community posts*

Auth	PUBLIC
Query	`postType?` (NEWS/RECRUIT/GENERAL), `siGunGuCode?`, `organizationId?`, pagination
Response	`List<CommunityPostResponse>` with `PageMeta`

GET `/communities/posts/{id}`

> *게시글 상세 / Get community post*

Auth	PUBLIC
Response	`CommunityPostResponse`

POST `/communities/posts`

> *게시글 작성 / Create community post*

Auth	USER
Request Body	`CreateCommunityPostRequest { title, content, postType, organizationId?, siGunGuCode? }`
Response	`CommunityPostResponse`
Status	201 Created

PUT `/communities/posts/{id}`

> *게시글 수정 / Update community post*

Auth	USER (author)
Response	`CommunityPostResponse`

DELETE `/communities/posts/{id}`

> *게시글 삭제 / Delete community post (soft delete)*

Auth	USER (author) / ADMIN
Status	204 No Content

PUT `/communities/posts/{id}/pin`

> *게시글 고정 / Pin post (manager only)*

Auth	USER (manager)

PUT `/communities/posts/{id}/unpin`

> *게시글 고정 해제 / Unpin post*

Auth	USER (manager)

POST `/communities/posts/{id}/report`

> *게시글 신고 / Report post*

Auth	USER
Request Body	`ReportPostRequest { category, reason }`
Response	`PostReportResponse`
Status	201 Created

GET `/communities/moderation/pending`

> *대기 중 신고 목록 / Pending reports (moderator)*

Auth	USER (moderator)
Query	`organizationId`, pagination

Response	`List<PostReportResponse>` with `PageMeta`

POST `/communities/moderation/reports/{id}/resolve`

> *신고 처리 / Resolve report*

Auth	USER (moderator)
Request Body	`ResolveReportRequest { resolution, note? }`
Response	`PostReportResponse`

POST `/communities/moderation/posts/{id}/action`

> *관리 조치 / Take moderation action*

Auth	USER (moderator)
Request Body	`ModerationActionRequest { actionType, reason }`
Response	`ModerationActionResponse`

5.22 Search

GET `/search`

> *통합 검색 / Unified search*

Auth	PUBLIC
Query	`q` (required), `types?` (comma-separated: LIVE,VOD,CLIP,TEAM,COMPETITION)
Response	`UnifiedSearchResponse { results[], totalCount }`

GET `/search/suggest`

> *검색 자동완성 / Search suggestions*

Auth	PUBLIC
Query	`q`
Response	`List<SearchSuggestion>`

GET `/search/trending`

> *인기 검색어 / Trending search terms*

Auth	PUBLIC
Response	`TrendingSearchResponse { terms[] }`

5.23 Schedule

GET `/schedule/today`

> *오늘의 대회 / Today's competitions*

Auth	PUBLIC
Query	`sportId?, month?`
Response	`List<TodayCompetitionItem>`

GET `/schedule/matches`

> *경기 일정 / Match schedule by date*

Auth	PUBLIC
Query	`sportId?, competitionId?, month?, date?`
Response	`ScheduleResponse { todayCompetitions[], matchesByDate }`

5.24 Recommendations

GET `/recommendations/personalized`

> *개인화 추천 / Personalized content recommendations*

Auth	USER
Response	`List<RecommendedContentResponse>`

GET `/recommendations/similar/{contentId}`

> *유사 콘텐츠 / Similar content*

Auth	PUBLIC
Response	`List<RecommendedContentResponse>`

GET `/recommendations/trending`

> *인기 콘텐츠 / Trending content*

Auth	PUBLIC
Response	`List<RecommendedContentResponse>`

GET `/recommendations/feed`

> *개인화 피드 / Personalized feed*

Auth	USER
Query	`page, size`
Response	`List<RecommendedContentResponse>`

GET `/recommendations/content-based/{contentId}`

> *콘텐츠 기반 추천 / Content-based recommendations*

Auth	PUBLIC
Query	`limit` (default: 10)
Response	`List<RecommendedContentResponse>`

5.25 Watch History

GET `/users/me/watch-history`

> *시청 기록 / Get watch history*

Auth	USER
Query	pagination
Response	`List<WatchHistoryResponse>` with `PageMeta`

POST `/users/me/watch-history`

> *시청 기록 저장 / Record watch event*

Auth	USER
Request Body	`RecordWatchEventRequest { contentType, contentId, watchDurationSec, lastPositionSec }`
Response	`WatchHistoryResponse`
Status	201 Created

5.26 Favorites

GET `/users/me/favorites`

> *즐겨찾기 목록 / Get favorites*

Auth	USER
Query	pagination
Response	`List<FavoriteResponse>` with `PageMeta`

POST `/users/me/favorites`

> *즐겨찾기 추가 / Add favorite*

Auth	USER
Request Body	`AddFavoriteRequest { targetType, targetId }`
Response	`FavoriteResponse`
Status	201 Created

DELETE `/users/me/favorites/{id}`

> *즐겨찾기 삭제 / Remove favorite*

Auth	USER

5.27 Notifications

POST `/notifications`

> *알림 생성 / Create notifications*

Auth	ADMIN
Request Body	`CreateNotificationRequest { userIds[], type, title, body, data? }`
Response	`List<NotificationResponse>`
Status	201 Created

POST `/notifications/internal`

> *내부 알림 생성 / Internal notification (service-to-service)*

Auth	INTERNAL
Request Body	`CreateNotificationRequest`
Status	201 Created

GET `/notifications`

> *내 알림 목록 / Get user notifications*

Auth	USER
Query	pagination
Response	`List<NotificationResponse>` with `PageMeta`

PUT `/notifications/{id}/read`

> *알림 읽음 처리 / Mark notification as read*

Auth	USER
Response	`NotificationResponse`

GET `/notifications/unread-count`

> *읽지 않은 알림 수 / Get unread notification count*

Auth	USER
Response	`{ "unreadCount": Long }`

5.28 Upload

POST `/upload/ticket`

> *업로드 티켓 생성 / Create upload ticket (presigned URL)*

Auth	USER
Request Body	`CreateUploadRequest { fileName, fileSizeBytes, contentType }`
Response	`UploadTicket { id, presignedUrl, expiresAt }`
Status	201 Created

POST `/upload/ticket/{id}/confirm`

> *업로드 확인 / Confirm upload complete, start transcoding*

Auth	USER
Response	`VodProcessingJob { id, status, progress }`

GET `/upload/jobs/{id}`

> *처리 상태 조회 / Get processing job status*

Auth	USER
Response	`VodProcessingJob`

POST `/upload/live-to-vod`

> *라이브 녹화본 VOD 변환 / Convert live recording to VOD*

Auth	ADMIN
Request Body	`LiveToVodRequest { liveAssetId, title?, description? }`
Response	`VodProcessingJob`
Status	201 Created

GET `/upload/vod/{vodAssetId}/playback`

> *VOD 재생 정보 / Get VOD playback info after processing*

Auth	USER
Response	`VodPlaybackInfo { hlsUrl, dashUrl?, duration, qualities[] }`

---

6. Commerce Service

Base path (via Gateway): `/api/v1` Internal port: 8083

6.1 Products

GET `/products`

> *상품 목록 / List products*

Auth	PUBLIC

Query	`productType?` (SUBSCRIPTION/SINGLE/POINT_PACK), `isActive?`, pagination
Response	`Page<ProductResponse>`

GET `/products/{id}`

> *상품 상세 / Product detail*

Auth	PUBLIC
Response	`ProductResponse { id, name, productType, priceKrw, pricePoint, durationDays, referenceType, referenceId, isActive }`

POST `/products`

> *상품 생성 / Create product*

Auth	ADMIN
Request Body	`CreateProductRequest { name, productType, priceKrw, pricePoint?, durationDays?, referenceType?, referenceId?, isActive }`
Response	`ProductResponse`
Status	201 Created

PUT `/products/{id}`

> *상품 수정 / Update product*

Auth	ADMIN
Response	`ProductResponse`

DELETE `/products/{id}`

> *상품 삭제 / Delete product*

Auth	ADMIN
Response	`null` with message "Product deleted"

6.2 Purchases

POST `/purchases`

> *구매 생성 / Create purchase*

Auth	USER
Request Body	`PurchaseRequest { productId, pgType, amount, couponId? }`
Response	`PurchaseResponse { id, userId, productId, pgType, amount, status, createdAt }`
Status	201 Created

GET `/purchases`

> *구매 내역 / List purchases*

Auth	USER
Query	pagination
Response	`Page<PurchaseResponse>`

GET `/purchases/{id}`

> *구매 상세 / Purchase detail*

Auth	USER
Response	`PurchaseResponse`

PUT `/purchases/{id}/cancel`

> *구매 취소 / Cancel purchase*

Auth	USER
Response	`PurchaseResponse` (status: CANCELLED)

6.3 Entitlements

GET `/entitlements/check`

> *시청 권한 확인 / Check entitlement*

Auth	USER
Query	`type?` (SUBSCRIPTION/SINGLE_PURCHASE), `scopeType?`, `scopeId?`
Response	`EntitlementCheckResponse { hasAccess, entitlementType?, expiresAt? }`

GET `/entitlements`

> *보유 권한 목록 / List active entitlements*

Auth	USER
Response	`List<EntitlementResponse>`

6.4 Wallet

GET `/wallet`

> *지갑 조회 / Get wallet balance*

Auth	USER
Response	`WalletResponse { id, userId, balance }`

GET `/wallet/history`

> *지갑 내역 / Wallet transaction history*

Auth	USER
Query	`ledgerType?` (CHARGE/USE/REFUND/EXPIRE), `dateFrom?`, `dateTo?`, pagination
Response	`Page<WalletHistoryResponse>`

POST `/wallet/charge`

> *포인트 충전 / Charge wallet*

Auth	USER
Request Body	`ChargeRequest { amount, pgType, pgTransactionId? }`
Response	`WalletResponse`

POST `/wallet/use`

> *포인트 사용 / Use points*

Auth	USER
Request Body	`UsePointsRequest { amount, referenceType?, referenceId?, description? }`
Response	`WalletResponse`

6.5 Refunds

POST `/refunds`

> *환불 신청 / Create refund request*

Auth	USER
Request Body	`RefundRequest { purchaseId, reason }`
Response	`RefundResponse { id, purchaseId, status, reason, createdAt }`
Status	201 Created

GET `/refunds`

> *환불 목록 / List refunds*

Auth	ADMIN
Query	`status?` (PENDING/APPROVED/REJECTED), pagination
Response	`Page<RefundResponse>`

PUT `/refunds/{id}/process`

> *환불 처리 / Process refund (approve/reject)*

Auth	ADMIN
Request Body	`ProcessRefundRequest { action, reason? }`

Response	`RefundResponse`

6.6 Coupons

GET `/coupons/my`

> *내 쿠폰 목록 / Get my coupons*

Auth	USER
Query	`status?` (AVAILABLE/USED/EXPIRED)
Response	`List<CouponResponse>`

POST `/coupons/register`

> *쿠폰 등록 / Register coupon by code*

Auth	USER
Request Body	`{ "code": string }`
Response	`CouponResponse`

POST `/coupons/{id}/use`

> *쿠폰 사용 / Use coupon*

Auth	USER
Response	`CouponResponse`

GET `/coupons/available`

> *사용 가능 쿠폰 / List available public coupons*

Auth	PUBLIC
Response	`List<CouponResponse>`

---

7. Operation Service

Base path (via Gateway): `/api/v1` Internal port: 8084

7.1 Venues

GET `/venues/search`

> *시설 검색 / Search venues*

Auth	PUBLIC
Query	`keyword?`, `sportId?`, `siGunGuCode?`, `venueType?`, `ownerType?`, pagination
Response	`List<VenueSearchResponse>` with `PageMeta`

GET `/venues/nearby`

> *주변 시설 / Nearby venues*

Auth	PUBLIC
Query	`lat` (BigDecimal, required), `lng` (BigDecimal, required), `radiusDegree?`, pagination
Response	`List<VenueSearchResponse>` with `PageMeta`

GET `/venues`

> *시설 목록 / List venues*

Auth	PUBLIC
Query	`ownerType?`, `venueType?`, `sportId?`, `name?`, pagination
Response	`List<VenueListResponse>` with `PageMeta`

GET `/venues/{id}`

> *시설 상세 / Venue detail*

Auth	PUBLIC
Response	`VenueDetailResponse`

POST `/venues`

> *시설 생성 / Create venue*

Auth	ADMIN
Request Body	`CreateVenueRequest`
Response	`VenueResponse`
Status	201 Created

PUT `/venues/{id}`

> *시설 수정 / Update venue*

Auth	ADMIN
Response	`VenueResponse`

DELETE `/venues/{id}`

> *시설 삭제 / Delete venue*

Auth	ADMIN
Status	204 No Content

POST `/venues/{id}/cameras`

> *카메라 연결 / Link camera to venue*

Auth	ADMIN
Request Body	`LinkCameraRequest { cameraId, isMain? }`
Response	`VenueDetailResponse`
Status	201 Created

DELETE `/venues/{id}/cameras/{cameraId}`

> *카메라 연결 해제 / Unlink camera from venue*

Auth	ADMIN
Status	204 No Content

7.2 Cameras

GET `/cameras`

> *카메라 목록 / List cameras*

Auth	ADMIN
Query	`name?`
Response	`List<CameraListResponse>`

GET `/cameras/{id}`

> *카메라 상세 / Camera detail*

Auth	ADMIN
Response	`CameraDetailResponse`

POST `/cameras`

> *카메라 생성 / Create camera*

Auth	ADMIN
Request Body	`CreateCameraRequest { name, cameraType?, productType?, serialNumber?, isPanorama? }`
Response	`CameraDetailResponse`
Status	201 Created

PUT `/cameras/{id}`

> *카메라 수정 / Update camera*

Auth	ADMIN
Response	`CameraDetailResponse`

DELETE `/cameras/{id}`

> *카메라 삭제 / Delete camera*

Auth	ADMIN
Status	204 No Content

7.3 Reservations

POST `/reservations`

> *촬영 예약 생성 / Create reservation*

Auth	USER
Request Body	`CreateReservationRequest { venueId, matchId?, startAt, endAt, purpose?, memo? }`
Response	`ReservationResponse`
Status	201 Created

GET `/reservations`

> *예약 목록 / List reservations*

Auth	USER / ADMIN
Query	`venueId?`, `status?`, `dateFrom?`, `dateTo?`, `reservedByUserId?`, pagination
Response	`List<ReservationResponse>` with `PageMeta`

GET `/reservations/{id}`

> *예약 상세 / Reservation detail*

Auth	USER
Response	`ReservationResponse`

PUT `/reservations/{id}/status`

> *예약 상태 변경 / Change reservation status (PENDING -> CONFIRMED/CANCELLED)*

Auth	ADMIN
Request Body	`ChangeStatusRequest { status, reason? }`
Response	`ReservationResponse`

GET `/reservations/calendar`

> *예약 캘린더 / Calendar view*

Auth	PUBLIC
Query	`mode` (month/week, default: month), `date` (ISO date, required), `venueId?`

Response	`List<CalendarEventResponse>`

7.4 Studio Sessions

GET `/studio/sessions`

> *스튜디오 세션 목록 / List studio sessions*

Auth	ADMIN
Query	`venueId?`, `matchId?`, pagination
Response	`Page<StudioSession>`

7.5 Streaming Ingest

POST `/streaming/ingest/endpoints`

> *인제스트 엔드포인트 생성 / Create RTMP ingest endpoint*

Auth	ADMIN
Request Body	`CreateIngestRequest { name, venueId?, rtmpUrl? }`
Response	`IngestEndpoint`
Status	201 Created

GET `/streaming/ingest/endpoints`

> *활성 인제스트 목록 / List active ingest endpoints*

Auth	ADMIN
Response	`List<IngestEndpoint>`

GET `/streaming/ingest/endpoints/{id}/status`

> *인제스트 상태 / Get ingest endpoint status*

Auth	ADMIN
Response	`IngestStatus { isReceiving, bitrate, uptime }`

POST `/streaming/ingest/transcode/start`

> *트랜스코딩 시작 / Start transcoding pipeline (RTMP -> HLS)*

Auth	ADMIN
Request Body	`StartTranscodeRequest { ingestEndpointId, config }`
Response	`TranscodeSession`
Status	201 Created

POST `/streaming/ingest/transcode/{id}/stop`

> *트랜스코딩 중지 / Stop transcoding*

Auth	ADMIN

GET `/streaming/ingest/playback/{sessionId}`

> *재생 URL 조회 / Get playback URLs for transcode session*

Auth	ADMIN
Response	`PlaybackInfo { hlsUrl, dashUrl? }`

---

8. Admin Service

Base path (via Gateway): `/admin/api/v1` (no prefix stripping) Internal port: 8085

8.1 Auth

POST `/admin/api/v1/auth/login`

> *BO 로그인 / Admin login*

Auth	PUBLIC
Request Body	`AdminLoginRequest { loginId, password }`
Response	`AdminLoginResponse { accessToken, refreshToken, name, role }`
Status	200 OK

8.2 RBAC - Roles

GET `/admin/api/v1/rbac/roles`

> *역할 목록 / List roles*

Auth	ADMIN
Response	`List<AdminRoleResponse>`

GET `/admin/api/v1/rbac/roles/{id}`

> *역할 상세 / Role detail*

Auth	ADMIN
Response	`AdminRoleResponse { id, name, code, description, displayOrder, isActive }`

POST `/admin/api/v1/rbac/roles`

> *역할 생성 / Create role*

Auth	ADMIN
Response	`AdminRoleResponse`

Status	201 Created

PUT `/admin/api/v1/rbac/roles/{id}`

> *역할 수정 / Update role*

Auth	ADMIN
Response	`AdminRoleResponse`

DELETE `/admin/api/v1/rbac/roles/{id}`

> *역할 삭제 / Delete role*

Auth	ADMIN

PUT `/admin/api/v1/rbac/roles/{id}/menus`

> *역할에 메뉴 할당 / Assign menus to role*

Auth	ADMIN
Request Body	`AssignMenusRequest { menuIds[] }`

PUT `/admin/api/v1/rbac/roles/{id}/functions`

> *역할에 기능 할당 / Assign functions to role*

Auth	ADMIN
Request Body	`AssignFunctionsRequest { functionIds[] }`

GET `/admin/api/v1/rbac/roles/{id}/menus`

> *역할의 메뉴 트리 / Get role menu tree*

Auth	ADMIN
Response	`List<AdminMenuTreeResponse>`

GET `/admin/api/v1/rbac/roles/{id}/functions`

> *역할의 기능 목록 / Get role functions*

Auth	ADMIN
Response	`List<AdminFunctionResponse>`

8.3 RBAC - Groups

GET `/admin/api/v1/rbac/groups`

> *그룹 트리 / Group tree*

Auth	ADMIN

Response	`List<AdminGroupTreeResponse>`

GET `/admin/api/v1/rbac/groups/{id}`

> *그룹 상세 / Group detail*

Auth	ADMIN
Response	`AdminGroupTreeResponse`

POST `/admin/api/v1/rbac/groups`

> *그룹 생성 / Create group*

Auth	ADMIN
Status	201 Created

PUT `/admin/api/v1/rbac/groups/{id}`

> *그룹 수정 / Update group*

Auth	ADMIN

DELETE `/admin/api/v1/rbac/groups/{id}`

> *그룹 삭제 / Delete group*

Auth	ADMIN

POST `/admin/api/v1/rbac/groups/{id}/members`

> *그룹에 멤버 할당 / Assign members to group*

Auth	ADMIN
Request Body	`AssignMembersRequest { memberIds[] }`

DELETE `/admin/api/v1/rbac/groups/{id}/members`

> *그룹에서 멤버 제거 / Remove members from group*

Auth	ADMIN
Request Body	`AssignMembersRequest { memberIds[] }`

POST `/admin/api/v1/rbac/groups/{id}/roles`

> *그룹에 역할 할당 / Assign roles to group*

Auth	ADMIN
Request Body	`AssignRolesRequest { roleIds[] }`

DELETE `/admin/api/v1/rbac/groups/{id}/roles`

> *그룹에서 역할 제거 / Remove roles from group*

Auth	ADMIN

GET `/admin/api/v1/rbac/groups/{id}/permissions`

> *유효 권한 목록 / Get effective permissions*

Auth	ADMIN
Response	`List<String>` (permission codes)

GET `/admin/api/v1/rbac/groups/{id}/members`

> *그룹 멤버 목록 / Get group members*

Auth	ADMIN
Response	`List<AdminUserListResponse>`

GET `/admin/api/v1/rbac/groups/{id}/roles`

> *그룹 역할 목록 / Get group roles*

Auth	ADMIN
Response	`List<AdminRoleResponse>`

8.4 RBAC - Menus

GET `/admin/api/v1/rbac/menus`

> *메뉴 트리 / Menu tree*

Auth	ADMIN
Response	`List<AdminMenuTreeResponse>`

GET `/admin/api/v1/rbac/menus/{id}`

> *메뉴 상세 / Menu detail*

Auth	ADMIN
Response	`AdminMenuTreeResponse`

POST `/admin/api/v1/rbac/menus`

> *메뉴 생성 / Create menu*

Auth	ADMIN
Request Body	`CreateMenuRequest { parentId?, name, menuType?, pageUrl?, icon?, displayOrder }`

Status	201 Created

PUT `/admin/api/v1/rbac/menus/{id}`

> *메뉴 수정 / Update menu*

Auth	ADMIN

DELETE `/admin/api/v1/rbac/menus/{id}`

> *메뉴 삭제 / Delete menu*

Auth	ADMIN

PUT `/admin/api/v1/rbac/menus/reorder`

> *메뉴 순서 변경 / Reorder menus*

Auth	ADMIN
Request Body	`ReorderMenuRequest { orders: [{id, displayOrder}] }`

8.5 RBAC - Functions

GET `/admin/api/v1/rbac/functions`

> *기능 목록 / List functions*

Auth	ADMIN
Response	`List<AdminFunctionResponse>`

GET `/admin/api/v1/rbac/functions/{id}`

> *기능 상세 / Function detail*

Auth	ADMIN

POST `/admin/api/v1/rbac/functions`

> *기능 생성 / Create function*

Auth	ADMIN
Request Body	`CreateFunctionRequest { code, name?, controller?, action? }`
Status	201 Created

PUT `/admin/api/v1/rbac/functions/{id}`

> *기능 수정 / Update function*

Auth	ADMIN

DELETE `/admin/api/v1/rbac/functions/{id}`

> *기능 삭제 / Delete function*

Auth	ADMIN

8.6 RBAC - Members (Admin Users)

GET `/admin/api/v1/rbac/members`

> *관리자 목록 / List admin users*

Auth	ADMIN
Query	pagination
Response	`Page<AdminUserListResponse>`

GET `/admin/api/v1/rbac/members/{id}`

> *관리자 상세 / Admin user detail*

Auth	ADMIN
Response	`AdminUserListResponse`

POST `/admin/api/v1/rbac/members`

> *관리자 생성 / Create admin user*

Auth	ADMIN
Request Body	`CreateAdminUserRequest { loginId, password, name, phone?, email? }`
Status	201 Created

PUT `/admin/api/v1/rbac/members/{id}`

> *관리자 수정 / Update admin user*

Auth	ADMIN

DELETE `/admin/api/v1/rbac/members/{id}`

> *관리자 삭제 / Delete admin user*

Auth	ADMIN

PATCH `/admin/api/v1/rbac/members/{id}/block`

> *관리자 차단 / Block admin user*

Auth	ADMIN
Response	`AdminUserListResponse`

PATCH `/admin/api/v1/rbac/members/{id}/unblock`

> *관리자 차단 해제 / Unblock admin user*

Auth	ADMIN
Response	`AdminUserListResponse`

8.7 Site - Banners

GET `/admin/api/v1/site/banners`

> *배너 목록 / List banners*

Auth	ADMIN
Response	`List<Banner>` (sorted by sortOrder)

GET `/admin/api/v1/site/banners/{id}`

> *배너 상세 / Banner detail*

Auth	ADMIN
Response	`Banner`

POST `/admin/api/v1/site/banners`

> *배너 생성 / Create banner*

Auth	ADMIN
Request Body	`Banner { title, imageUrl, linkUrl?, sortOrder, isActive }`
Status	201 Created

DELETE `/admin/api/v1/site/banners/{id}`

> *배너 삭제 / Delete banner*

Auth	ADMIN
Status	204 No Content

8.8 Site - Notices

GET `/admin/api/v1/site/notices`

> *공지사항 목록 / List notices*

Auth	ADMIN
Query	pagination
Response	`Page<Notice>` (pinned first, then by createdAt desc)

GET `/admin/api/v1/site/notices/{id}`

> *공지사항 상세 / Notice detail*

Auth	ADMIN
Response	`Notice`

POST `/admin/api/v1/site/notices`

> *공지사항 생성 / Create notice*

Auth	ADMIN
Status	201 Created

DELETE `/admin/api/v1/site/notices/{id}`

> *공지사항 삭제 / Delete notice*

Auth	ADMIN
Status	204 No Content

8.9 CS - Inquiries

GET `/admin/api/v1/cs/inquiries`

> *문의 목록 / List inquiries*

Auth	ADMIN
Query	`status?`, pagination
Response	`Page<Inquiry>`

GET `/admin/api/v1/cs/inquiries/{id}`

> *문의 상세 / Inquiry detail*

Auth	ADMIN
Response	`Inquiry`

8.10 CS - Reports

GET `/admin/api/v1/cs/reports`

> *신고 목록 / List reports*

Auth	ADMIN
Query	`status?`, pagination
Response	`Page<Report>`

GET `/admin/api/v1/cs/reports/{id}`

> *신고 상세 / Report detail*

Auth	ADMIN
Response	`Report`

8.11 CS - Terms

GET `/admin/api/v1/cs/terms`

> *약관 목록 / List terms*

Auth	ADMIN
Response	`List<Term>` (active only)

GET `/admin/api/v1/cs/terms/{id}`

> *약관 상세 / Term detail*

Auth	ADMIN
Response	`Term`

POST `/admin/api/v1/cs/terms`

> *약관 생성 / Create term*

Auth	ADMIN
Status	201 Created

DELETE `/admin/api/v1/cs/terms/{id}`

> *약관 삭제 / Delete term*

Auth	ADMIN
Status	204 No Content

8.12 App Versions

GET `/admin/api/v1/app/versions`

> *앱 버전 목록 / List app versions*

Auth	ADMIN
Query	`platform?` (AOS/IOS)
Response	`List<AppVersion>`

GET `/admin/api/v1/app/versions/latest`

> *최신 버전 / Get latest version*

Auth	ADMIN

Query	`platform` (required)
Response	`AppVersion`

POST `/admin/api/v1/app/versions`

> *앱 버전 생성 / Create app version*

Auth	ADMIN
Status	201 Created

DELETE `/admin/api/v1/app/versions/{id}`

> *앱 버전 삭제 / Delete app version*

Auth	ADMIN
Status	204 No Content

8.13 Analytics

POST `/admin/api/v1/analytics/events`

> *이벤트 수집 / Bulk event ingestion*

Auth	PUBLIC (from mobile/web clients)
Request Body	`BulkEventRequest { events: [{ eventType, userId?, contentId?, properties, timestamp }] }`
Response	`{ "ingested": Integer }`

GET `/admin/api/v1/analytics/dashboard`

> *대시보드 통계 / Dashboard aggregated stats*

Auth	ADMIN
Response	`DashboardStatsResponse { totalVisitors, totalViews, revenue, topContent[], activeUsersTrend[] }`

---

9. Gateway

Port: 8080

9.1 Health

GET `/health`

> *게이트웨이 상태 / Gateway health*

Auth	PUBLIC
Response	`{ "service": "pochak-gateway", "status": "UP", "timestamp": "..." }`

GET `/health/services`

> *전체 서비스 상태 / All services health*

Auth	PUBLIC
Response	`Map<String, ServiceHealth>` where `ServiceHealth { serviceName, status, url, responseTimeMs, checkedAt }`

9.2 Rate Limiting

The Gateway applies rate limiting at two tiers:

Route Pattern	Limit	Window
`/api/v1/auth/**`	10 requests	per minute
All other `/api/v1/**`	100 requests	per minute

Rate limiting uses Redis when available (`pochak.rate-limit.type=redis`), with automatic fallback to in-memory ConcurrentHashMap. Response header `X-RateLimit-Remaining` is included. HTTP 429 is returned when exceeded.

---

10. Policy Compliance Matrix

This matrix maps API endpoints to POCHAK_POLICY.md requirements.

10.1 CUG (Closed User Group) Access Control

Policy Requirement	Implementing Endpoints	Notes
CUG 단체: 콘텐츠 직접 노출 불가, 회원만 접근	`GET /contents/{type}/{id}/access`, `GET /contents/{type}/{id}/stream`	VideoAclService evaluates ABAC rules per content
홍보용 영상 비회원도 열람 가능	`GET /contents/{type}/{id}/access`	Visibility=PUBLIC bypasses membership check
OPEN 단체: 자유 가입	`POST /memberships/join`	Auto-APPROVED for OPEN orgs
CLOSED 단체: 관리자 승인 필요	`POST /memberships/join`, `PUT /memberships/{id}/approve`	PENDING until manager approval

10.2 ABAC Video Permission

Policy Requirement	Implementing Endpoints	Notes
Visibility: PUBLIC/MEMBERS_ONLY/PRIVATE	`POST /contents/{type}/{id}/acl`, `GET /contents/{type}/{id}/acl`	SetAclRequest sets visibility level
조직 멤버십 기반 접근	`GET /contents/{type}/{id}/access`	VideoAclService checks membership table
구독권 기반 접근	`GET /entitlements/check`, `GET /contents/{type}/{id}/stream`	Commerce entitlement checked during stream access

10.3 Subscription & Purchase

Policy Requirement	Implementing Endpoints	Notes
상품 구매 -> 권한 부여	`POST /purchases` -> `GET /entitlements`	Purchase creates entitlement automatically
포인트(뽈) 충전/사용	`POST /wallet/charge`, `POST /wallet/use`	Wallet ledger tracks all transactions
쿠폰 등록/사용	`POST /coupons/register`, `POST /coupons/{id}/use`	Coupon applies discount at purchase

환불 정책	`POST /refunds`, `PUT /refunds/{id}/process`	Admin processes refund, entitlement revoked

10.4 Guardian & Minor Protection

Policy Requirement	Implementing Endpoints	Notes
미성년자 월간 결제 한도	`PUT /guardians/{id}/limit`, `GET /guardians/check-limit`	Commerce calls check-limit before purchase
보호자 인증 후 가입	`POST /auth/signup/minor`, `POST /auth/guardian/verify`	Guardian consent token required
보호자 관계 해제	`DELETE /guardians/{id}`	Revokes guardian-minor link

10.5 Security (SEC-001 ~ SEC-006)

SEC ID	Description	Implementation
SEC-001	Strip untrusted X-User-Id/X-User-Role	`JwtValidationFilter` strips headers before processing
SEC-002	Token blacklist (logout/block)	Redis key `token-blacklist:{userId}`, checked on every request
SEC-003	PKCE for mobile OAuth	`OAuth2Controller.initiateOAuth()` validates code_challenge, `/token` verifies code_verifier
SEC-004	Admin route protection	`JwtValidationFilter` rejects non-ADMIN role on `/api/v1/admin/**`
SEC-005	Rate limiting	Auth: 10/min, General: 100/min via Redis or in-memory
SEC-006	Auth code instead of URL tokens	One-time 30s auth code via `AuthCodeStore`, exchanged at `/auth/oauth2/token`

10.6 User Withdrawal

Policy Requirement	Implementing Endpoints	Notes
회원 탈퇴 시 전 서비스 데이터 정리	`DELETE /auth/withdraw`	Publishes `UserWithdrawnEvent` via RabbitMQ, all services listen and clean up
탈퇴 후 토큰 무효화	`DELETE /auth/withdraw`	Token blacklisted in Redis

End of API Specification